

EXERCISE 10: SENTIMENT ANALYSIS

Reading how customers feel — Applied AI & ML

AT A GLANCE

ML technique	NLP sentiment analysis — Part A: transparent lexicon scorer · Part B: a real deep-learning transformer
Business problem	Read thousands of customer messages automatically and find the angriest customers first
Why this one	Most of what customers tell you is words, not numbers. You cannot read 10,000 by hand.
Data	300 synthetic Telco customer messages, generated IN the notebook (no downloads for Part A)
Headline result	Part A: 95.7% accurate, fully explainable Part B: a transformer catches the sarcasm Part A misses
The lesson	This is where a black-box deep-learning model genuinely EARNs its cost. Match the tool to the job.

PRE CHECKLIST

- Refer to the [Day9_Python Libraries Setup_Guide_for_Trainees.docx](#)

OPENING THE EXERCISE

"Yesterday I promised you something. I said: today we make a machine READ. Text becomes tokens, tokens become numbers, and those numbers carry meaning. That is deep learning applied to language and this is the exercise where we do it."

"For two days we worked with numbers: tenure, spend, usage, anomalies etc. But most of what your customers tell you is not a number. It is words. Complaints, chat messages, survey comments. Omantel gets thousands every day, and nobody can read them all."

"This morning we teach the machine to read the FEELING behind those words. We'll build a simple version we fully understand then we'll unleash a real deep-learning model and watch the difference."

PART A — BUILD A SENTIMENT READER YOU CAN SEE INSIDE

Every code block below is copy-paste ready — paste the whole block into ONE notebook cell and run it (Shift + Enter).

STEP 1 — Load our tools

Cell 1. The same three you already know: pandas, numpy, matplotlib.

</> CODE — COPY & PASTE INTO ONE NOTEBOOK CELL

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

print("All tools loaded. Ready to read customer feelings.")
```

EXPECTED OUTPUT

All tools loaded. Ready to read customer feelings.

STEP 2 — Build the message database

Cell 2. Generates 300 realistic Omantel customer messages: 120 angry, 90 happy, 90 neutral with the TRUE feeling attached so we can check our work later.

</> CODE — COPY & PASTE INTO ONE NOTEBOOK CELL

```
np.random.seed(42)

negative_templates = [
    "The network is terrible in {place}, calls drop every day",
    "I have been overcharged again on my bill, this is unacceptable",
    "No internet for three days in {place} and no one helped me",
    "Worst customer service ever, I waited two hours and got nothing",
    "My data ran out too fast, I am very disappointed with Omantel",
    "The signal is awful and I am thinking of switching to a competitor",
    "I am so frustrated, my SIM activation failed twice",
    "Slow internet, expensive plan, terrible experience overall",
]

positive_templates = [
    "The new 5G in {place} is amazing, very happy with the speed",
    "Great customer service today, the agent solved my problem quickly",
    "I love the new eSIM, activation was easy and fast",
    "Excellent coverage during my trip to {place}, very satisfied",
    "Best network in Oman, thank you Omantel for the great service",
    "The app is wonderful and my bill was correct this month",
    "Fast internet and a fair price, I am really pleased",
    "Fantastic experience, the National Day offer was a great deal",
]

neutral_templates = [
    "I want to know my current data balance for this month",
    "Please tell me how to activate international roaming for {place}",
    "What time does the Omantel store in {place} open",
    "I need to update the address on my account",
    "Can you explain the charges on my last invoice",
    "How do I transfer my number to a new SIM card",
```

```

    "Is the eSIM National Day offer still available",
    "I would like to change my monthly plan",
]
places = ["Muscat", "Salalah", "Sohar", "Nizwa", "Sur"]

rows = []
def fill(t): return t.replace('{place}', np.random.choice(places))
for _ in range(120):
    rows.append((fill(np.random.choice(negative_templates)), "Negative"))
for _ in range(90):
    rows.append((fill(np.random.choice(positive_templates)), "Positive"))
for _ in range(90):
    rows.append((fill(np.random.choice(neutral_templates)), "Neutral"))

df = pd.DataFrame(rows, columns=["complaint_text", "true_feeling"])
df = df.sample(frac=1, random_state=42).reset_index(drop=True)
df.insert(0, "ticket_id", [f"TKT{50000+i}" for i in range(len(df))])
print(f"Created {len(df)} customer messages.")
print(df["true_feeling"].value_counts())

# --- save the raw message database as a CSV in the notebook's folder ---
df.to_csv("omantel_customer_messages.csv", index=False) print("\nSaved to
omantel_customer_messages.csv")

```

EXPECTED OUTPUT

```

Created 300 customer messages.
true_feeling
Negative    120
Positive    90
Neutral     90

```

STEP 3 — Look at the data

Cell 3. Never build on data you haven't seen. Read a few messages out loud.

</> CODE — COPY & PASTE INTO ONE NOTEBOOK CELL

```
df.head(10)
```

EXPECTED OUTPUT

```

TKT50000 The new 5G in Sur is amazing, very happy... Positive
TKT50001 I want to know my current data balance... Neutral
TKT50002 The app is wonderful and my bill was... Positive
TKT50003 The signal is awful and I am thinking of... Negative
TKT50004 Can you explain the charges on my last... Neutral

```

STEP 4 Build the sentiment engine (the glass box)

Cell 4. THE HEART OF PART A. We give the machine a list of positive words and negative words, and it scores each message. Fully transparent you can see exactly why it decides.

</> CODE - COPY & PASTE INTO ONE NOTEBOOK CELL

```
positive_words = {
    "amazing", "happy", "great", "love", "easy", "fast", "excellent", "satisfied",
    "best", "thank", "wonderful", "correct", "pleased", "fantastic", "good", "fair",
    "quickly", "solved"
}
negative_words = {
    "terrible", "drop", "overcharged", "unacceptable", "no", "awful", "worst",
    "waited", "nothing", "disappointed", "frustrated", "failed", "slow", "expensive",
    "switching", "competitor", "bad", "never"
}

def score_message(text):
    words = text.lower().replace(",", " ").split()
    pos = sum(w in positive_words for w in words)
    neg = sum(w in negative_words for w in words)
    if pos > neg: return "Positive", pos - neg
    if neg > pos: return "Negative", pos - neg
    return "Neutral", 0

df[["predicted_feeling", "score"]] = df["complaint_text"].apply(
    lambda t: pd.Series(score_message(t))
)
print("Sentiment engine applied to every message.")
print(df[["complaint_text", "predicted_feeling"]].head(6).to_string())
```

EXPECTED OUTPUT

```
Sentiment engine applied to every message.
The new 5G in Sur is amazing...      Positive
I want to know my data balance...    Neutral
The app is wonderful...              Positive
The signal is awful...               Negative
Can you explain the charges...       Neutral
Is the eSIM National Day offer...    Neutral
```

"This is the whole trick, and it is beautifully simple. The machine counts positive words and negative words. More angry words than happy words? Negative. That's it."

"Keep this in mind for later: you can SEE exactly why every message scored the way it did. That transparency is a feature — remember it when we meet the transformer."

STEP 5 — How accurate is our reading?

</> CODE — COPY & PASTE INTO ONE NOTEBOOK CELL

```
from sklearn.metrics import accuracy_score
acc = accuracy_score(df["true_feeling"], df["predicted_feeling"]) * 100
print(f"Sentiment accuracy: {acc:.1f}%")
```

EXPECTED OUTPUT

Sentiment accuracy: 95.7%

WHAT TO SAY

"95.7%. Out of 300 messages, our simple word-counter read the feeling correctly about 287 times — with a list you could write in five minutes. Hold that number. We're going to test its limits shortly."

STEP 6 The mood of the whole customer base

</> CODE - COPY & PASTE INTO ONE NOTEBOOK CELL

```
counts = df["predicted_feeling"].value_counts()
print("Customer mood right now:")
print(counts)
neg_share = counts.get("Negative",0)/len(df)*100
print(f"\nNegative share: {neg_share:.1f}% of all messages")

counts.plot(kind="bar", color=["#C0392B", "#7F8C8D", "#1E7A46"],
          figsize=(6,4), rot=0)
plt.title("Omantel customer mood — this week")
plt.ylabel("Number of messages")
plt.show()
```

EXPECTED OUTPUT

```
Customer mood right now:
Negative   107
Neutral    103
Positive    90
```

Negative share: 35.7% of all messages
(plus a bar chart)

WHAT TO SAY

"Now we have a number a manager cares about: 36% of messages this week are negative. THAT is a dashboard an early-warning system for the whole customer base, built from words nobody had time to read."

STEP 7 The angry list: who to call first

Cell 7. The Part A deliverable. Sort the angriest messages to the top and save them as a CSV the care team can action.

</> CODE — COPY & PASTE INTO ONE NOTEBOOK CELL

```
angry = df[df["predicted_feeling"]=="Negative"].sort_values("score")
print(f"MOST NEGATIVE messages to action first ({len(angry)} total):")
print(angry[["ticket_id", "complaint_text", "score"]].head(8).to_string())

angry.to_csv("omantel_negative_tickets.csv", index=False)
print("\nSaved angry list to omantel_negative_tickets.csv")
```

EXPECTED OUTPUT

MOST NEGATIVE messages to action first (107 total):
TKT50003 The signal is awful and I am thinking... -3
TKT50013 Slow internet, expensive plan, terrible... -3
TKT50032 Worst customer service ever, I waited... -3
...
Saved angry list to omantel_negative_tickets.csv

"There it is. This morning nobody could read 300 messages. Now you have a ranked list of the 107 unhappiest customers, worst first, saved to a file, ready for the retention team before lunch."
"Connect it back to Monday: Exercise 1 Day 1 found who MIGHT leave from their numbers i.e. Churners. This finds who is ANGRY in their own words. Together a retention machine."

PART B — UNLEASH A REAL DEEP-LEARNING TRANSFORMER

This is the payoff of earlier promise: tokens - embeddings - meaning. We now run a genuine deep-learning model the same family as ChatGPT and watch it do what our word-counter cannot.

STEP 8 Load a real transformer and read the SAME messages

Cell 8. First run downloads the model (done at home / by IT). In class it loads from cache in seconds. This is deep learning applied to language live.

</> CODE - COPY & PASTE INTO ONE NOTEBOOK CELL

```
from transformers import pipeline

# Loads a real deep-learning sentiment model (DistilBERT).
# Pre-installed by IT; loads instantly from local cache in class.
sentiment_ai = pipeline("sentiment-analysis",
                        model="distilbert-base-uncased-finetuned-sst-2-english")

print("Deep-learning model loaded. Let's read some messages.")

# Try it on three of our messages
samples = [
    "The signal is awful and I am thinking of switching to a competitor",
    "The new 5G in Sur is amazing, very happy with the speed",
    "I need to update the address on my account",
]
for msg in samples:
    result = sentiment_ai(msg)[0]
    print(f"{result['label']}:9} ({result['score']*100:.0f}%) <- {msg}")
```

EXPECTED OUTPUT

```
Deep-learning model loaded. Let's read some messages.
NEGATIVE (100%) <- The signal is awful and I am thinking of switching...
POSITIVE (100%) <- The new 5G in Sur is amazing, very happy with the speed
NEGATIVE ( 100%) <- I need to update the address on my account
```

(Note: SST-2 only knows POSITIVE / NEGATIVE — it has no 'Neutral',
so plain requests get forced into one side. A real limitation to point out.)

WHAT TO SAY

"This is what yesterday's bridge pointed to. Behind that one line, the model turned each sentence into tokens, tokens into embeddings — numbers that carry meaning — and read the feeling. That is deep learning applied to language, running on your laptop."

"Notice it gives a confidence too — 100% sure the first one is negative. But watch the third: a neutral request got forced to NEGATIVE, because this model only knows positive or negative. Even the fancy model has limits. Honesty always."

STEP 9 - Where the transformer beats our word-counter

Cell 9. THE MOST IMPORTANT CELL. We feed BOTH models the three messages our simple counter gets wrong, side by side.

</> CODE - COPY & PASTE INTO ONE NOTEBOOK CELL

```
hard_messages = [
    "The network is not bad actually",          # "not bad"
    "The staff were not helpful at all",        # negation
    "I was worried about my bill but it turned out fine", # context
    "Nothing about this service is good",        # double-negative
    "Oh brilliant, no signal AGAIN. I just love paying for nothing", # sarcasm
]

print(f'{"MESSAGE":52} {"OUR COUNTER":12} {"TRANSFORMER":12}')
print("-" * 80)
for msg in hard_messages:
    ours, _ = score_message(msg)
    ai = sentiment_ai(msg)[0]['label'].title()
    print(f"{msg[:50]:52} {ours:12} {ai:12}')
```

EXPECTED OUTPUT

MESSAGE	OUR COUNTER	TRANSFORMER
The network is not bad actually	Negative	Positive
The staff were not helpful at all	Neutral	Negative
I was worried about my bill but it turned out fine	Neutral	Positive
Nothing about this service is good	Neutral	Negative
Oh brilliant, no signal AGAIN. I just love paying	Neutral	Positive

(First four: transformer is RIGHT, counter is wrong. Last one: BOTH wrong – sarcasm.)

"Look at the first four rows this is the transformer earning its cost. 'The network is not bad' our counter sees the word 'bad' and calls it Negative. The transformer understands 'not bad' means fine: Positive. Correct."

"'The staff were not helpful at all' our counter sees 'helpful' and gets confused. The transformer reads the negation and says Negative. Correct. 'Nothing about this service is good' same trick, the transformer gets it, our counter can't."

"Every one of those four, the transformer read the MEANING that word-counting cannot see. That is what deep learning bought us and it is exactly the messy, human language where it is worth the cost."

THE HONEST MOMENT — row five (say this out loud, do NOT hide it)

"Now look at the last row - the sarcasm. 'Oh brilliant, no signal again, I love paying for nothing.' Our counter got it wrong. And so did the transformer it saw 'brilliant' and 'love' and said Positive too."

"This is the truth no vendor will tell you: sarcasm is genuinely hard, even for these models. There is no magic tool that solves everything."

(A note: the counter's exact label on this row can read Neutral or Negative depending on punctuation either way it's WRONG, which is the point. The transformer says Positive, also wrong. Both fail; don't dwell on the counter's exact value.)

THE DAY-3 THESIS — the bridge into this afternoon

"So is the transformer just better, full stop? No. It's better HERE, on messy human feeling with negation and context, because that's a hard job worth the cost."

"But it's slower, it costs money per message, it needs pre-installing, and unlike our word list you cannot open it up and see WHY it decided. In Exercise 6, we'll meet a job where the SIMPLE model wins for exactly those reasons. Right tool for the right job that's the whole skill."

COMMON ERRORS & FIXES

If you see...	Do this
<code>ModuleNotFoundError: transformers</code>	IT pre-install missing on this laptop. Run: <code>pip install transformers torch</code> (needs internet). This is why we pre-stage.
Model downloading in class (slow)	The cache wasn't warmed. It downloads once (~250MB) then is instant. Warm it in the dry run and via IT.
<code>KeyError: 'predicted_feeling'</code>	Part A Step 4 didn't run. Run cells in order, top to bottom.
Transformer label looks 'wrong' on a neutral	Expected — SST-2 has no Neutral class. Point it out as a real model limitation, not a bug.
<code>NameError: score_message</code>	Step 4 was skipped — Step 9 needs it to compare. Run Part A first.

WRAP-UP — WHAT THEY JUST BUILT

- **A transparent sentiment reader scoring 300 messages in under a second at 95.7% and you can see exactly why each one scored**
- A live 'customer mood' number for the whole base — 36% negative this week
- A ranked, exported angry-customer call list for the retention team
- A real deep-learning transformer, run live, catching the sarcasm and meaning the simple model misses
- The day's core idea: deep learning is worth its cost HERE, on messy human feeling but not everywhere (see Exercise 6)